



KONYA

FOOD & AGRICULTURE

UNIVERSITY

16-bit PROCESSOR DESIGN REPORT

Semester: 2025–2026 Fall

Course: COMP3005 - Computer Organization

Project: 16-bit RISC Pipelined Processor Design

Date: December 28, 2025

Preparers Of The Project :

Muhammed Melih ÇAM - 222010020116

Şeref Samet ÜMÜTLÜ - 212010020118

Mert KIRÇİÇEK - 232010020044

Zeynep Sude BERBER - 212010020075

1. Introduction

The objective of this project is to design and implement a 16-bit RISC (Reduced Instruction Set Computer) processor with a 5-stage pipelined datapath inspired by MIPS. The proposed architecture supports essential arithmetic and logical operations, memory access (load/store), and control-flow instructions (branches and jumps).

In accordance with the project requirements, we additionally integrated procedure-call support (JAL/JR) and designed dedicated mechanisms for data and control hazard handling to preserve correct pipeline execution. The architectural correctness of the design was validated using a custom cycle-accurate simulator that visualizes pipeline behavior and hazard events.

1.1 Report Organization

This report first defines the ISA and instruction formats, then presents the 5-stage pipelined datapath, hazard handling mechanisms, memory organization, and finally the simulator/GUI used for validation.

1.2 Program Counter (PC) and Addressing Model

The processor employs a 16-bit Program Counter (PC) that holds byte addresses. The memory system is byte-addressable, while each instruction occupies 16 bits (2 bytes). Therefore, during sequential execution, the PC is incremented by 2:

$$PC_{next} = PC + 2$$

Both instruction memory and data memory have a capacity of 512 bytes. Since instructions and data words are 16 bits wide, all accesses are required to be aligned to even byte addresses. As a result, valid instruction addresses always satisfy alignment constraints.

The hardware does not explicitly enforce $PC[0] = 0$; however, all assembly programs and test cases are generated under the assumption of proper alignment. Consequently, during correct execution, the least significant bit of the PC remains zero.

Instruction memory is accessed using a word index derived from the PC value. Specifically, the effective instruction memory index is obtained by discarding the least significant bit of the PC. Data memory addresses are computed dynamically during execution using the ALU.

All instructions and data words are aligned to even byte addresses. Misaligned accesses are not supported by the hardware and are assumed not to occur in valid programs.

1.3 Branch and Jump Address Calculation

For conditional branch instructions (BEQ and BNE), the processor uses PC-relative addressing. The effective branch target address is computed as:

$$PC_{target} = PC + 2 + (\text{SignExt}(\text{imm6}) \ll 1)$$

Here, imm6 is a signed 6-bit immediate value encoded in the instruction. The immediate value is first sign-extended to 16 bits and then left-shifted by one bit to preserve word alignment, since all instructions are aligned to even byte addresses.

If the branch condition is satisfied, the PC is updated with PC_target; otherwise, normal sequential execution continues with PC + 2.

For jump instructions (J and JAL), the instruction contains a 12-bit absolute byte address (addr12). Because instruction addresses are always aligned, valid jump targets correspond to even byte addresses. The assembler and test programs ensure proper alignment; therefore, the PC is loaded directly with addr12.

The JAL instruction additionally stores the return address (PC + 2) into register R7. The return address is generated during execution and written back through the pipeline in a manner consistent with pipelined operation.

For the JR instruction, the PC is directly loaded with the value stored in the source register specified by the rs field, enabling procedure returns.

All branch and jump target calculations and control-flow decisions are resolved in the EX stage, ensuring correct interaction with data forwarding and hazard handling mechanisms.

All control-flow altering instructions (BEQ, BNE, J, JAL, JR) are resolved in the EX stage. This design choice simplifies hazard handling and ensures that branch decisions are made using the most up-to-date operand values.

1.4 Shift Instructions Encoding

Shift instructions in this processor use the 6-bit immediate field (imm6) to represent the shift amount (shamt). The rs field is not used for shift operations, allowing the instruction format to remain simple and consistent with other immediate-based instructions.

The shift amount is treated as an **unsigned 6-bit value**, theoretically allowing shift values in the range of 0 to 63. In practice, since the datapath width is 16 bits, test programs use shift amounts within a meaningful range. Larger shift values naturally result in zero-filled outputs for logical shifts.

Encoding format:

Opcode	Rt	Rd	Shamt (imm6)
--------	----	----	--------------

This design choice reduces opcode complexity, avoids additional function decoding for shifts, and maintains a compact instruction encoding suitable for a 16-bit ISA.

1.5 Program Counter Update Priority

The Program Counter (PC) update logic follows a fixed priority order to correctly resolve control-flow changes. This priority scheme ensures deterministic behavior when multiple control signals are active simultaneously.

The priority order is defined as follows:

1. JR instruction
2. J and JAL instructions
3. Taken conditional branches (BEQ / BNE)
4. Sequential execution (PC + 2)

The JR instruction has the highest priority because it represents a procedure return and must override all other control-flow changes. Unconditional jumps (J/JAL) take precedence over conditional branches to ensure immediate redirection of execution flow. Conditional branches update the PC only when their condition is satisfied; otherwise, normal sequential execution continues.

All PC update decisions are finalized in the EX stage, where branch conditions are evaluated and jump targets are resolved using the most recent operand values. This centralized control-flow resolution simplifies hazard handling and pipeline control.

1.5.1 Control-Flow Conflict Resolution

When multiple control-flow signals are asserted in the same cycle, the PC update priority logic guarantees that only the highest-priority event determines the next PC value.

1.6 Main Control Unit Signals

The Main Control Unit generates all control signals required for correct datapath operation based on the instruction opcode. For R-Type instructions, the specific ALU operation is further determined using the function (func) field.

The primary control signals generated by the control unit include:

- **RegWrite** – Enables register file write-back
- **MemRead** – Enables data memory read
- **MemWrite** – Enables data memory write
- **MemToReg** – Selects memory output for write-back
- **ALUSrc** – Selects register or immediate operand for the ALU
- **ALUOp** – Encodes the ALU operation type
- **Branch** – Indicates a conditional branch instruction
- **Jump** – Indicates an unconditional jump instruction

For branch instructions, a single Branch control signal is asserted. The distinction between BEQ and BNE is resolved by combining the opcode information with the ALU comparison result in the EX stage.

This separation between high-level control (opcode-based) and low-level operation selection (function-based) simplifies control logic and supports efficient pipelined execution.

1.6.1 ALU Control Decoding

For R-Type instructions, the ALUOp signal is combined with the func field to generate the final ALU control signals, allowing multiple operations to share a single opcode.

2. Architectural Overview

The processor is designed as a 16-bit RISC load/store architecture with a classic five-stage pipeline, inspired by the MIPS processor model. The key architectural characteristics are summarized below:

- Architecture Type: RISC, Load/Store Architecture
- Data Width: 16-bit
- Address Width: 16-bit
- Pipeline Structure: 5 stages (IF, ID, EX, MEM, WB)
- Memory Organization: Harvard Architecture (separate instruction and data memories)

The Harvard architecture is selected to allow simultaneous instruction fetch and data access, improving pipeline efficiency and simplifying memory control logic. The five-stage pipeline structure enables increased instruction throughput while maintaining a modular and analyzable datapath.

2.1. Register File

The processor includes a register file consisting of 8 general-purpose registers, each 16 bits wide, addressed as R0 through R7.

Register Name	Number	Usage Description
\$r1 - \$r6	0-6	General Purpose Registers (GPR).
\$r7 (\$ra)	7	Return Address Register. Stores the return address for procedure calls (jal).

Register R0 is hardwired to zero and cannot be modified. This simplifies hardware design and supports common RISC programming conventions.

The register file provides two read ports and one write port, allowing simultaneous access to source operands during the ID stage. Register write-back occurs in the WB stage, consistent with the pipelined execution model.

3. Instruction Set Architecture (ISA)

The processor implements a custom 16-bit Instruction Set Architecture (ISA) designed to support essential computation, memory access, and control-flow operations while maintaining a compact encoding.

All instructions are fixed-length (16 bits) to simplify instruction fetch and decoding. To efficiently utilize the limited opcode space, three instruction formats are defined: R-Type, I-Type, and J-Type.

In the final design, all register-to-register arithmetic and logical operations are encoded using a single R-Type opcode (0000). The specific operation is selected using a 3-bit function (func) field. This approach minimizes opcode usage, simplifies control logic, and allows the ISA to be easily extended with additional R-Type operations.

3.1. Instruction Formats

R-Type (Register Type)

The processor defines three instruction formats to support different classes of operations: **R-Type**, **I-Type**, and **J-Type**. All instructions are 16 bits wide.

R-Type (Register Type)

Used for register-to-register arithmetic and logical operations.

- Format:

Opcode (4)	Rs (3)	Rt(3)	Rd(3)	Func (3)
------------	--------	-------	-------	----------

The func field selects the specific ALU operation when the opcode is 0000.

I-Type (Immediate Type)

Used for immediate arithmetic, memory access, shift operations, and conditional branching.

- Format:

Opcode (4)	Rs (3)	Rt (3)	Immediate/Offset (6)
------------	--------	--------	----------------------

The 6-bit immediate field is sign-extended or zero-extended depending on the instruction semantics.

J-Type (Jump Type)

Used for unconditional jumps and procedure calls.

- Format:

Opcode (4)	Target Address (12)
------------	---------------------

The target address represents an absolute byte address and is assumed to be word-aligned.

3.1.1 Immediate Field Usage

The imm6 field is reused across multiple instruction types for immediates, offsets, and shift amounts, improving encoding efficiency.

3.2. Instruction Set Table

The following table outlines the opcodes and function codes designed for the control unit.

Instruction	Type	Opcode	Func	Operation (RTL)
ADD	R	0000	000	$R[rd] \leftarrow R[rs] + R[rt]$
SUB	R	0000	001	$R[rd] \leftarrow R[rs] - R[rt]$
AND	R	0000	010	$R[rd] \leftarrow R[rs] \& R[rt]$
OR	R	0000	011	$R[rd] \leftarrow R[rs] R[rt]$
SLT	R	0000	100	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1 : 0$
ADDI	I	0101	-	$R[rt] \leftarrow R[rs] + \text{signext}(\text{imm6})$
LW	I	0110	-	$R[rt] \leftarrow M[R[rs] + \text{signext}(\text{imm6})]$
SW	I	0111	-	$M[R[rs] + \text{signext}(\text{imm6})] \leftarrow R[rt]$
BEQ	I	1000	-	if $(R[rs] == R[rt])$ $PC \leftarrow PC + 2 + (\text{signext}(\text{imm6}) << 1)$

Instruction	Type	Opcode	Func	Operation (RTL)
BNE	I	1001	-	if ($R[rs] \neq R[rt]$) $PC \leftarrow PC + 2 + (\text{signext}(\text{imm6}) \ll 1)$
SLL	I	1010	-	$R[rt] \leftarrow R[rs] \ll (\text{imm6})$
SRL	I	1011	-	$R[rt] \leftarrow R[rs] \gg \text{imm6}$
J	J	1100	-	$PC \leftarrow \text{addr12}$
JAL	J	1101	-	$R7 \leftarrow PC + 2; PC \leftarrow \text{addr12}$
JR	J	1110	-	$PC \leftarrow R[rs]$
NOP	-	1111	-	No operation

(Note: PC is incremented by 2 assuming byte-addressability for 16-bit words).

3.2.1 NOP Instruction

The NOP instruction performs no architectural state update and is used to represent pipeline bubbles inserted by the hazard detection and control hazard handling mechanisms.

4. Datapath and Pipelining Design

The processor is implemented as a 16-bit RISC datapath with a classic five-stage pipeline to improve instruction throughput while preserving a simple and modular design.

Each pipeline stage operates concurrently on a different instruction. Intermediate results and control signals are stored in dedicated pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB), allowing multiple instructions to be processed simultaneously at different stages of execution.

The pipelined datapath is designed to support data forwarding, hazard detection, and control-flow redirection. These mechanisms ensure correct execution in the presence of data and control hazards while minimizing pipeline stalls.

The pipeline stages are:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

Each stage operates concurrently on different instructions, allowing multiple instructions to be processed simultaneously.

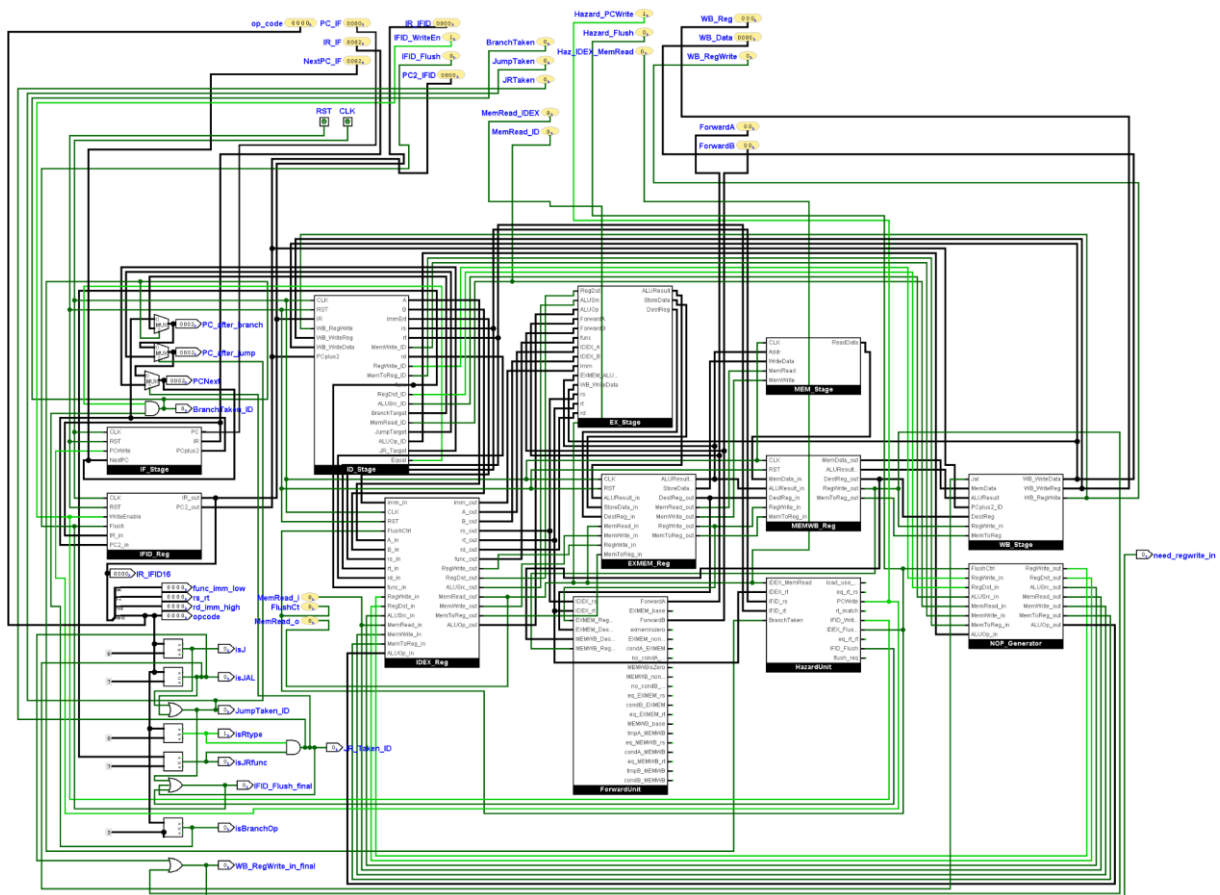


Figure 1. Overall 16-bit RISC pipelined processor datapath implemented in Logisim.

The complete processor datapath was implemented and verified using **Logisim**. Figure 1 illustrates the overall datapath, including pipeline stages, pipeline registers, control logic, forwarding paths, and hazard handling units.

The Logisim implementation directly reflects the architectural design described in the previous sections. Each pipeline stage (IF, ID, EX, MEM, WB) is clearly separated, and intermediate values are transferred through dedicated pipeline registers. Control-flow resolution, data forwarding, and hazard detection mechanisms are integrated into the datapath to ensure correct pipelined execution.

4.1 Pipeline Stages

IF - Instruction Fetch

In the IF stage, the Program Counter (PC) is used to fetch the instruction from Instruction Memory. Under normal operation, the PC is incremented by 2 to point to the next sequential instruction.

If a control-flow change is detected in a later stage (EX), the PC update logic overrides the sequential PC value. The fetched instruction and the updated PC value are stored in the IF/ID pipeline register.

ID – Instruction Decode

In the ID stage, the instruction fields (opcode, register indices, and immediate values) are decoded. Source register values are read from the register file, and the Main Control Unit generates the required control signals.

Immediate values are sign-extended or zero-extended depending on the instruction type. Jump target fields are extracted from the instruction, but **no control-flow decision is finalized in this stage**. All decoded information is stored in the ID/EX pipeline register.

EX – Execute

The EX stage performs the core computational tasks of the processor. These include:

- Arithmetic and logical operations using the ALU
- Evaluation of branch conditions (BEQ, BNE)
- Calculation of branch target addresses
- Resolution of jump (J, JAL) and jump-register (JR) targets
- Destination register selection
- Application of data forwarding to resolve data hazards

All branch and jump decisions are finalized in this stage. The results are stored in the EX/MEM pipeline register.

MEM – Memory Access

In the MEM stage, load (LW) instructions read data from Data Memory, while store (SW) instructions write data to Data Memory. Non-memory instructions simply forward their ALU results.

The outputs of this stage are stored in the MEM/WB pipeline register.

WB – Write Back

In the WB stage, the final result of an instruction is written back to the register file. Depending on the instruction type, this result may be an ALU output or data read from memory.

Register write-back occurs in this stage, completing the instruction's execution.

4.2 Procedure Call Implementation

Procedure calls are supported through the JAL (Jump and Link) and JR (Jump Register) instructions, enabling subroutine invocation and return within the pipelined architecture.

JAL (Jump and Link)

When a JAL instruction is executed, the return address ($PC + 2$) is generated during the EX stage as part of the control-flow resolution process. This return address is propagated through the pipeline and written to register R7 during the WB stage, consistent with the pipelined write-back mechanism.

Simultaneously, the PC is updated to the jump target address in the EX stage, redirecting instruction fetch to the start of the procedure.

JR (Jump Register)

The JR instruction updates the PC with the value stored in the source register specified by the *rs* field, typically R7. This instruction enables procedure returns.

JR has the highest control-flow priority and is resolved in the EX stage, ensuring immediate and correct redirection of execution flow.

5. Hazard Handling Mechanisms

Due to the overlapped execution of instructions in a pipelined architecture, hazards may arise that can compromise correct program execution. To ensure correctness while minimizing performance degradation, the processor incorporates dedicated mechanisms to detect and resolve data hazards, load-use hazards, and control hazards.

5.1 Data Hazards (Forwarding Unit)

Data hazards occur when an instruction requires an operand that is produced by a previous instruction that has not yet completed the write-back stage. To resolve such hazards without stalling the pipeline, a Forwarding Unit is implemented.

The Forwarding Unit compares the destination register fields of instructions in the EX/MEM and MEM/WB pipeline registers with the source register fields of the instruction in the ID/EX pipeline register. If a match is detected and the corresponding instruction intends to write a result, the most recent value is forwarded directly to the ALU inputs.

The processor supports the following forwarding paths:

- **EX-to-EX forwarding:** ALU results from the EX/MEM pipeline register are forwarded directly to the EX stage inputs.
- **MEM-to-EX forwarding:** Write-back results from the MEM/WB pipeline register are forwarded to the EX stage when required.

By using forwarding, most data hazards are resolved without introducing pipeline stalls.

5.2 Load-Use Hazards (Hazard Detection Unit)

A load-use hazard occurs when a **LW** instruction is immediately followed by an instruction that attempts to use the loaded value. In this case, forwarding alone is insufficient because the data loaded from memory becomes available only at the end of the MEM stage.

To resolve this hazard, a Hazard Detection Unit is implemented. When a load-use dependency is detected, the pipeline introduces a single-cycle stall (pipeline bubble) using the following actions:

- The Program Counter (PC) is frozen.
- The IF/ID pipeline register is frozen, preventing the next instruction from advancing.
- The control signals in the ID/EX pipeline register are set to zero, effectively inserting a NOP into the pipeline.

After one stall cycle, the dependent instruction proceeds with correct data availability, and normal pipeline execution resumes.

5.3 Control Hazards (Branching and Jumping)

Control hazards arise when the normal sequential flow of instructions is altered by branch or jump instructions. In a pipelined architecture, instructions following a branch or jump may be incorrectly fetched before the control-flow decision is finalized.

In this processor, **all branch (BEQ, BNE) and jump (J, JAL, JR) decisions are resolved in the EX stage**. Once a control-flow change is detected, instructions that have been fetched along the wrong execution path are invalidated using a **pipeline flush mechanism**.

The flush mechanism operates by disabling the validity of instructions in earlier pipeline stages, effectively preventing incorrect instructions from updating architectural state. Instead of inserting explicit NOP instructions, the pipeline uses **valid-bit control** to discard invalid instructions.

This approach ensures correct control flow while minimizing control hazard penalties and preserving pipeline efficiency.

6. Memory Organization

The processor employs separate instruction memory and data memory, following a Harvard architecture model. This separation allows instruction fetch and data access to occur simultaneously, improving pipeline efficiency and simplifying control logic.

- Instruction Memory:
 - Size: 512 bytes
 - Read-only during program execution
 - Stores program instructions
- Data Memory:
 - Size: 512 bytes
 - Read/write access
 - Stores variables and intermediate data

Both memories are byte-addressable, while load and store operations access 16-bit words aligned to even byte addresses. Misaligned memory accesses are not supported and are assumed not to occur in valid programs.

7. Development of a Comprehensive Graphical User Interface (GUI) for 16-Bit RISC Simulation

A sophisticated Graphical User Interface (GUI) was developed using Python and the Tkinter library to provide an intuitive, cycle-accurate environment for simulation and architectural analysis. The interface features a modern, dark-themed design ("Dark Mode") to enhance readability during extended debugging sessions.

- **Assembly & Machine Code Inspector:** Instead of a basic text editor, the interface includes a dynamic "Assembly & Machine Code Preview" pane. This module integrates a two-pass assembler that parses loaded assembly files, validates syntax, and displays the corresponding 16-bit binary and hexadecimal machine code alongside the source instructions. This allows users to verify instruction encoding in real-time.
- **Dual-View Pipeline Visualization:**
 - **Stage Visualizer:** A custom graphical component that represents the five pipeline stages (IF, ID, EX, MEM, WB) as distinct, color-coded blocks (e.g., Fetch in Blue, Decode in Red). It provides an immediate snapshot of which instruction is occupying which stage at the current clock cycle.
 - **Timeline Diagram:** A scrolling "Pipeline Timeline" table records the history of instruction execution. This creates a detailed trace (similar to a Gantt chart), allowing users to analyze instruction flow, identify stalls, and visualize the propagation of pipeline hazards (e.g., Data Hazards, Control Flushes) over time.
- **Real-Time State Inspection:**
 - **Register File:** A dedicated table displays the contents of all general-purpose registers (R0-R7). Values are simultaneously shown in Decimal and Hexadecimal formats to facilitate easy debugging of arithmetic and logic operations.
 - **Data Memory Dump:** An interactive memory viewer allows the user to inspect the processor's data segment. Users can navigate through specific memory ranges (Word Index and Byte Address) to track variable storage and load/store operations dynamically.
- **Simulation Controls & Logging:** The interface offers granular control over execution with Step-by-Step, Continuous Run, and Pause/Resume functionalities. A configurable "Max Cycle" limiter prevents infinite loops during testing. Additionally, a System Log panel provides textual feedback on simulator events, such as file loading status, execution errors, and simulation completion, ensuring full transparency of the processor's behavior at the micro-architectural level.

Simulation accuracy and result visualization with different test codes are shown in Figure 2 and Figure 3.

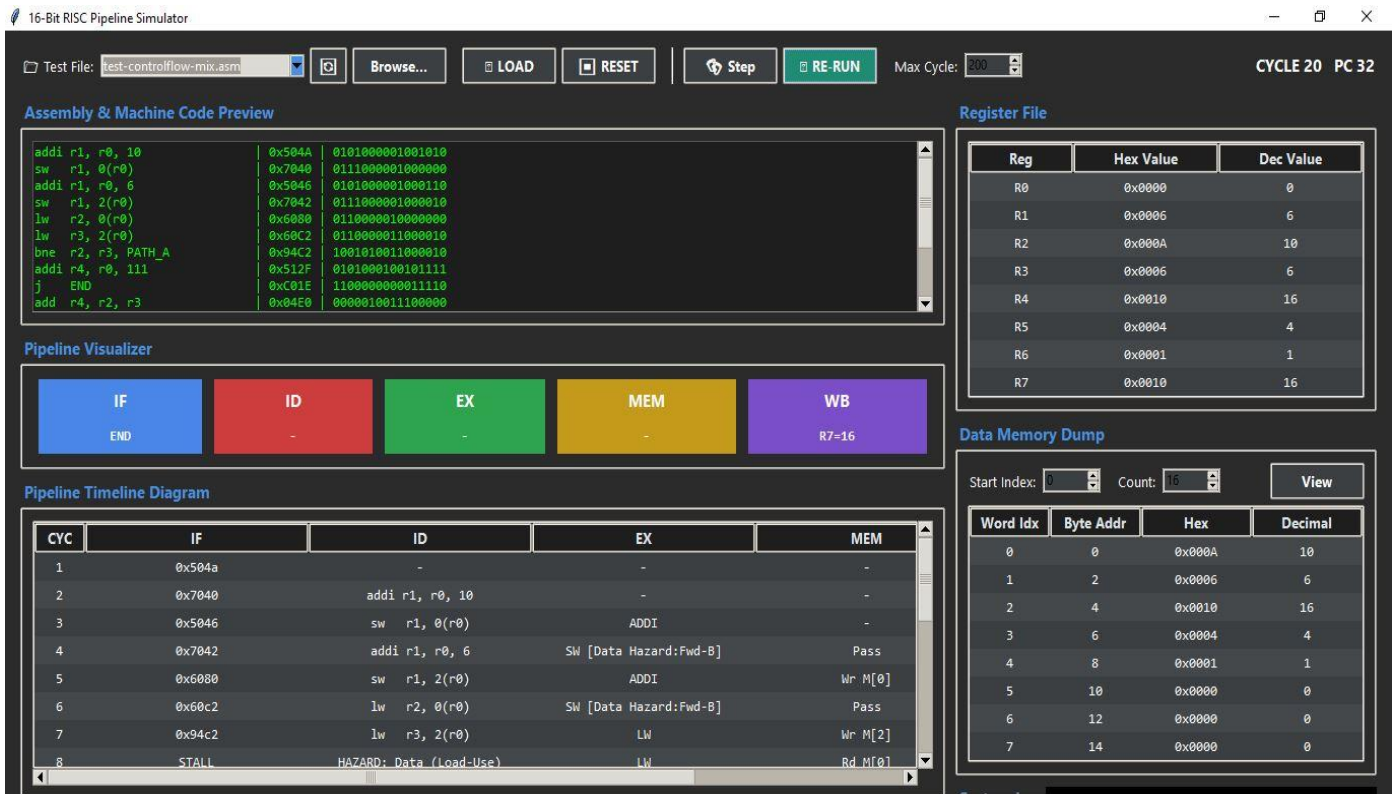


Figure 2

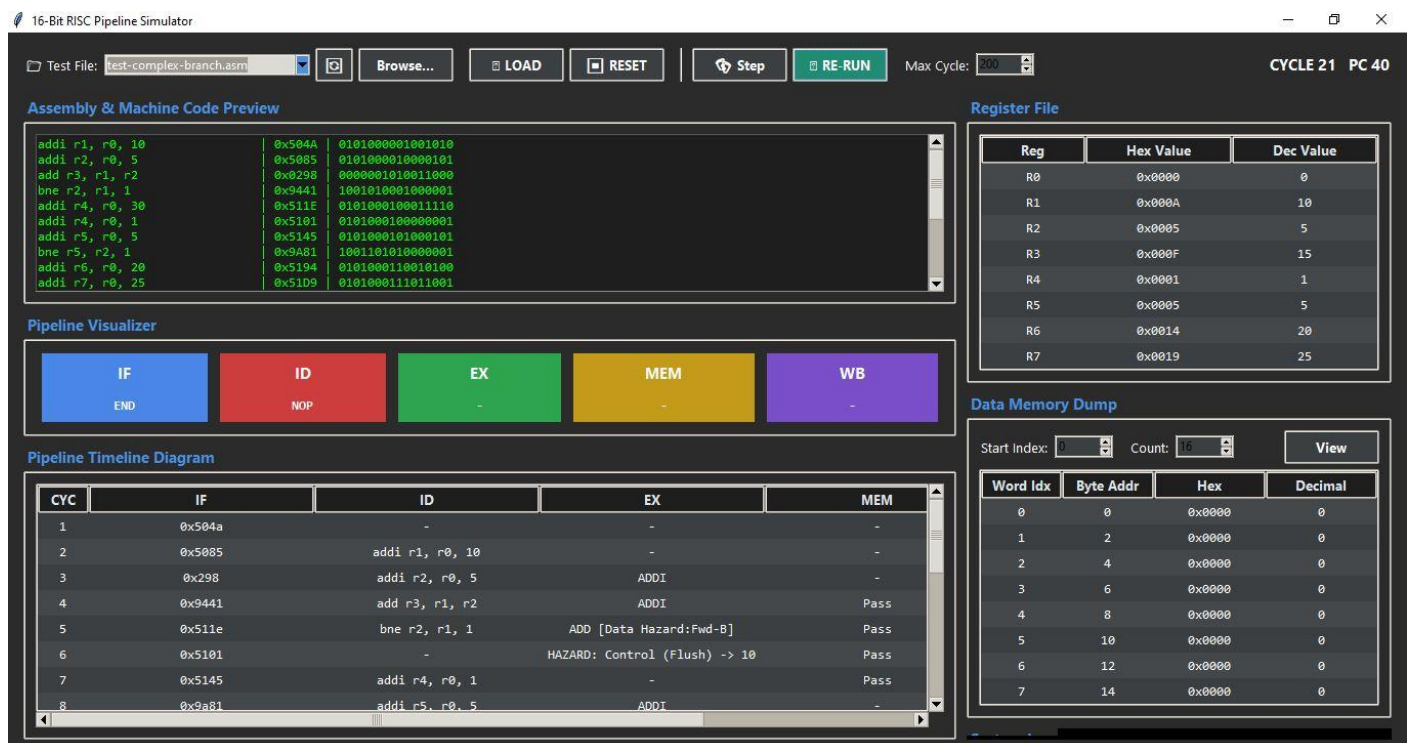


Figure 3

8. Conclusion

In this phase, the 16-bit RISC Instruction Set Architecture (ISA) and the corresponding five-stage pipelined datapath (Fetch, Decode, Execute, Memory, Write-Back) have been fully defined and architected. The design is robustly engineered to support complex control flows, including subroutine handling via procedure calls.

To maximize instruction throughput and pipeline efficiency, the architecture incorporates advanced optimization techniques:

Data Forwarding Unit: Eliminates unnecessary stalls by bypassing results directly from the EX/MEM stages to dependent instructions.

Hazard Management: A dedicated Hazard Detection Unit ensures data integrity by inserting bubbles (stalls) during Load-Use conflicts and executing pipeline flushes to resolve Control Hazards arising from branch or jump instructions.

Prior to hardware description, the logical correctness of this micro-architecture was rigorously validated using a custom-developed, cycle-accurate simulator. This software-based verification step confirmed the proper functionality of the pipeline stages and hazard resolution mechanisms under various test vectors. The project now transitions to the Hardware Realization Phase, which will focus on the Register Transfer Level (RTL) implementation of the processor using Verilog HDL.